

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Pruebas de Software				
TÍTULO DE LA PRÁCTICA:	Pruebas de Integración: API Testing con Postman y Supertest				
NÚMERO DE LA PRÁCTICA:	08	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	01/07/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Barrios Medina, Mathias Alonso - Cuno Salazar, Eduardo Joel - Hanco Mullisaca, Sergio Danilo - Sucle Suca, Michael Benjamin				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Profe. Robert Arisaca					

RESULTADOS Y PRUEBAS
Ejercicio 1: Automatización con Supertest Elijan una temática de su preferencia (ej. catálogo de videojuegos, gestión de una biblioteca musical, reserva de películas, etc.) e implementen una API REST básica en Node.js + Express. Posteriormente, diseñen y ejecuten una suite completa de pruebas de integración utilizando Supertest y Jest/Mocha. Requerimientos de la Prueba: - Flujo de Persistencia Cruzada: Validar la integración secuencial entre la creación de un recurso (POST /recurso) y su posterior lectura (GET /recurso/:id), asegurando que el ID generado dinámicamente en el primer endpoint sea el puente de entrada para el segundo. - Simulación de Modificación de Estado: Implementar y verificar un endpoint que altere una propiedad cuantitativa del recurso (ej. restar stock, cambiar nivel, actualizar reproducciones) y confirmar mediante un GET posterior que el cambio se consolidó en el mock de persistencia o base de datos de pruebas. - Validación de Robustez (Edge Cases): Asegurar que el sistema maneje correctamente los desajustes de datos o tipos inválidos (ej. enviar un texto en un campo numérico o dejar campos obligatorios vacíos) devolviendo exactamente el código de estado HTTP 400 (Bad Request) junto con su respectivo mensaje de error. Entregables: - Archivo de la API (app.js). - Archivo de pruebas (tema_libre.test.js). - Captura de pantalla o reporte en texto del resultado de la ejecución en la terminal usando Jest/Mocha. Desarrollo del Backend - Catálogo de Videojuegos I. Configuración del Proyecto Se creó el proyecto con Node.js y Express. Las dependencias utilizadas se muestran en el archivo <i>package.json</i>: <pre>{ "name": "videojuegos-api", "version": "1.0.0",</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

"description": "",
"main": "index.js",
"scripts": {
  "start": "node server.js",
  "dev": "node server.js",
  "test": "jest",
  "coverage": "jest --coverage"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs",
"devDependencies": {
  "jest": "^30.4.2",
  "supertest": "^7.2.2"
},
"dependencies": {
  "express": "^5.2.1"
}
}

```

II. Servidor Implementado

Se implementaron los siguientes endpoints en *app.js*:

- POST /videojuegos - Crear videojuego
- GET /videojuegos/:id - Leer videojuego
- PUT /videojuegos/:id/stock - Actualizar stock
- DELETE /videojuegos/:id - Eliminar videojuego
- DELETE /videojuegos - Limpiar todos (para pruebas)

El archivo *server.js* importa la aplicación desde *app.js* e inicia el servidor en el puerto 3000:

```

const app = require('./app');

// Puerto para el servidor local
const PORT = 3000;

app.listen(PORT, () => {
  console.log(`SERVIDOR DE VIDEOJUEGOS`);
  console.log(` Servidor corriendo en:`);
  console.log(` http://localhost:${PORT}`);
  console.log(` Endpoints disponibles:`);
  console.log(` POST /videojuegos - Crear videojuego`);
  console.log(` GET /videojuegos/:id - Leer videojuego`);
  console.log(` PUT /videojuegos/:id/stock - Actualizar stock`);
  console.log(` DELETE /videojuegos/:id - Eliminar videojuego`);
  console.log(` DELETE /videojuegos - Limpiar todos (para pruebas)`);
  console.log(`=====`);
  console.log(` Presiona Ctrl+C para detener el servidor`);
});

```

Validaciones en POST:

- Campos obligatorios: nombre, genero, stock, precio
- Tipos de datos: stock y precio numéricos
- Valores lógicos: stock y precio ≥ 0

III. Servidor en Ejecución

El servidor corre en `http://localhost:3000`.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```

~/UNSA/2026A/PS/lab_08/src/ejercicio-1 sdhm git:(main) 12974 • +586584 -583867 10:35 pm (19s)
npm start

> videojuegos-api@1.0.0 start
> node server.js

SERVIDOR DE VIDEOJUEGOS
Servidor corriendo en:
http://localhost:3000
Endpoints disponibles:
POST /videojuegos - Crear videojuego
GET /videojuegos/:id - Leer videojuego
PUT /videojuegos/:id/stock - Actualizar stock
DELETE /videojuegos/:id - Eliminar videojuego
DELETE /videojuegos - Limpiar todos (para pruebas)
=====
Presiona Ctrl+C para detener el servidor

```

Figura 1: Servidor ejecutándose en la terminal

IV. Pruebas Manuales con Postman

4.1. Crear Videojuego - POST

Se envió un POST a /videojuegos con un cuerpo JSON conteniendo los campos nombre, género, stock y precio. El servidor validó los datos, asignó un ID único automáticamente y respondió con 201 Created junto con el objeto del videojuego creado.

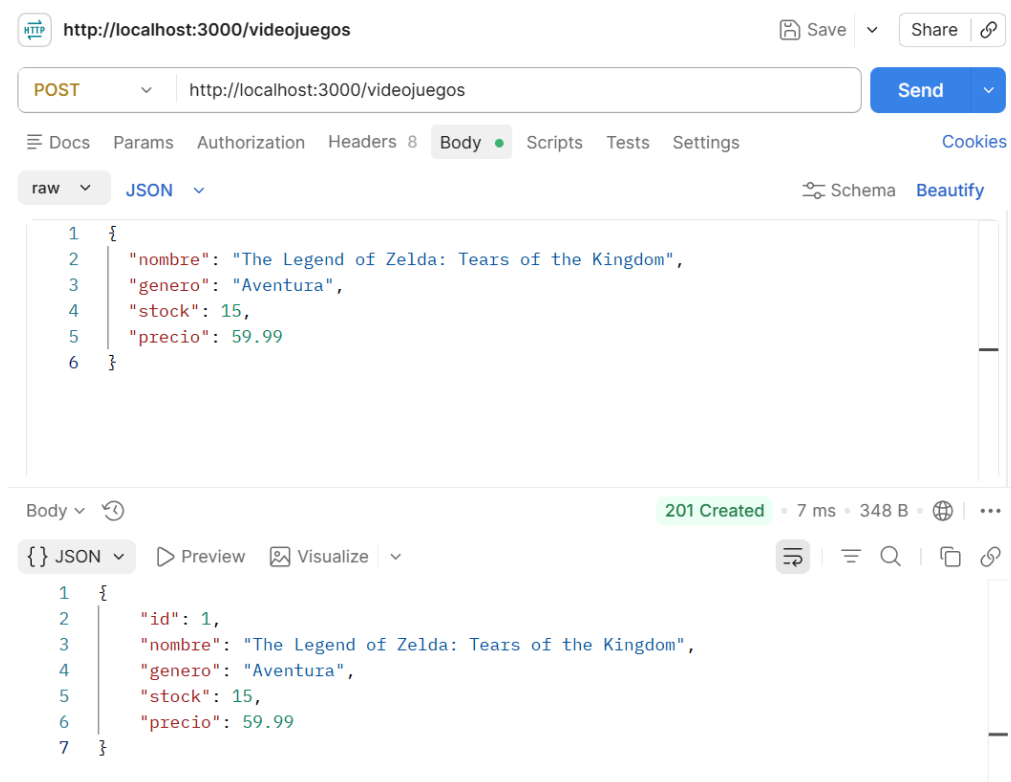


Figura 2: Resultado: Status 201, ID asignado: 1

4.2. Leer Videojuego - GET

A continuación consultamos el recurso recién creado haciendo un GET a /videojuegos/1 con el ID asignado. El servidor lo encontró en el arreglo en memoria y devolvió 200 OK con los datos completos del videojuego.

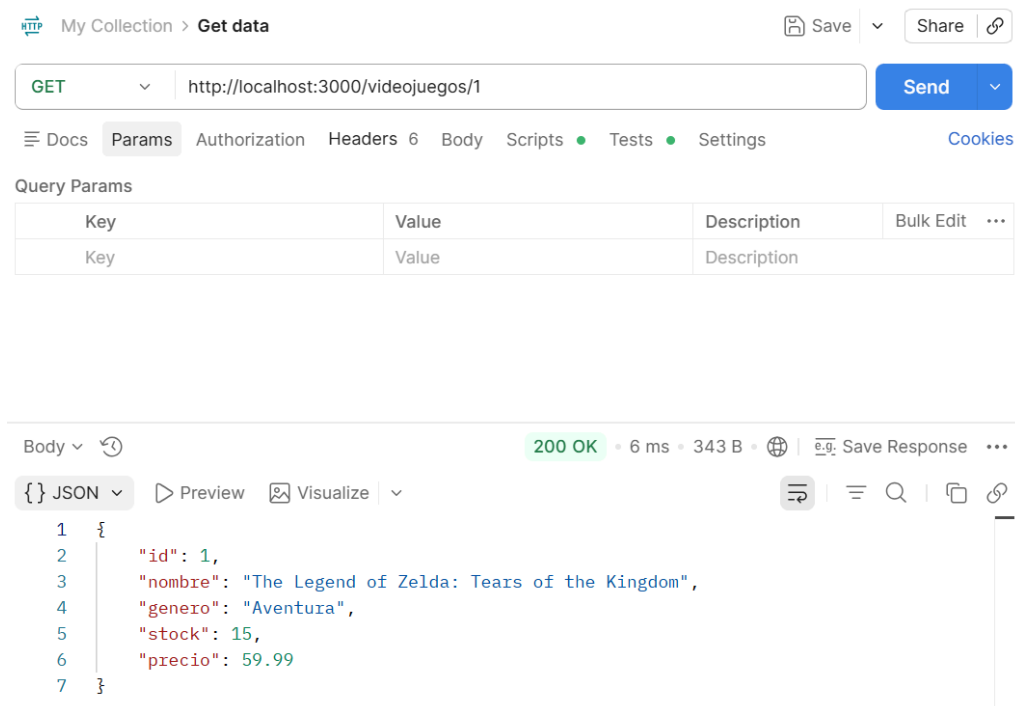


Figura 3: Resultado: Status 200, datos correctos

4.3. Actualizar Stock - PUT

Probamos modificar el stock del videojuego con un PUT a /videojuegos/1/stock enviando el nuevo valor en el cuerpo. El servidor validó que fuera numérico y no negativo, localizó el recurso por su ID y actualizó la propiedad. Respondió con 200 OK y el objeto con el stock modificado.

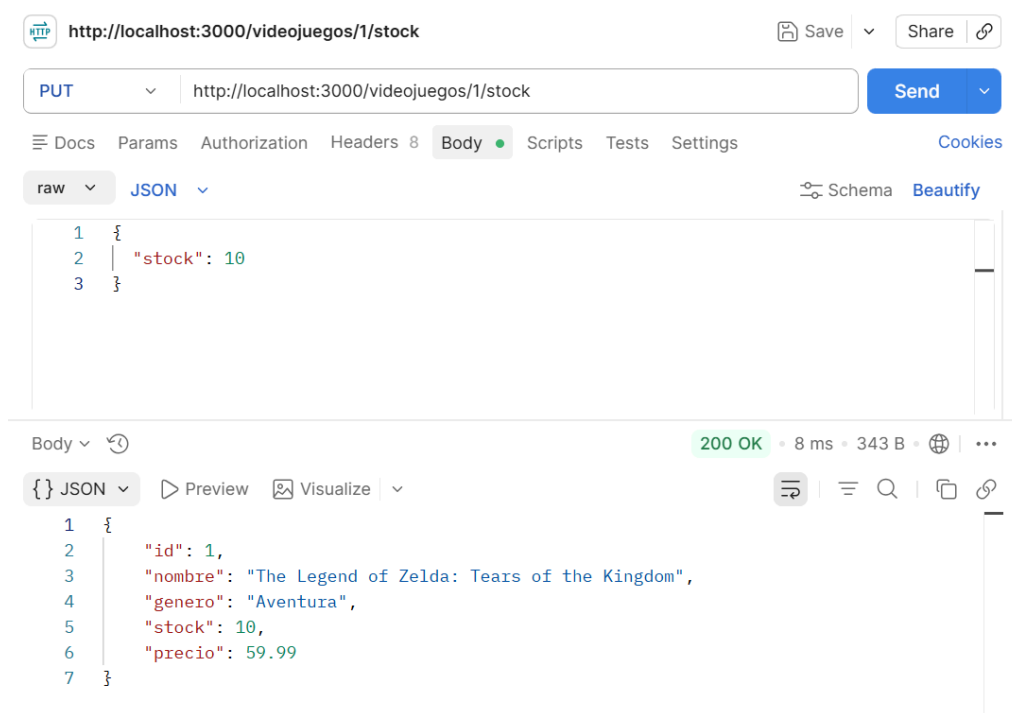


Figura 4: Resultado: Status 200, stock actualizado

4.4. Eliminar Videojuego - DELETE

Luego ejecutamos un DELETE a `/videojuegos/1`. El servidor buscó el videojuego por su ID, lo eliminó del arreglo en memoria y respondió con 200 OK junto con un mensaje de confirmación.

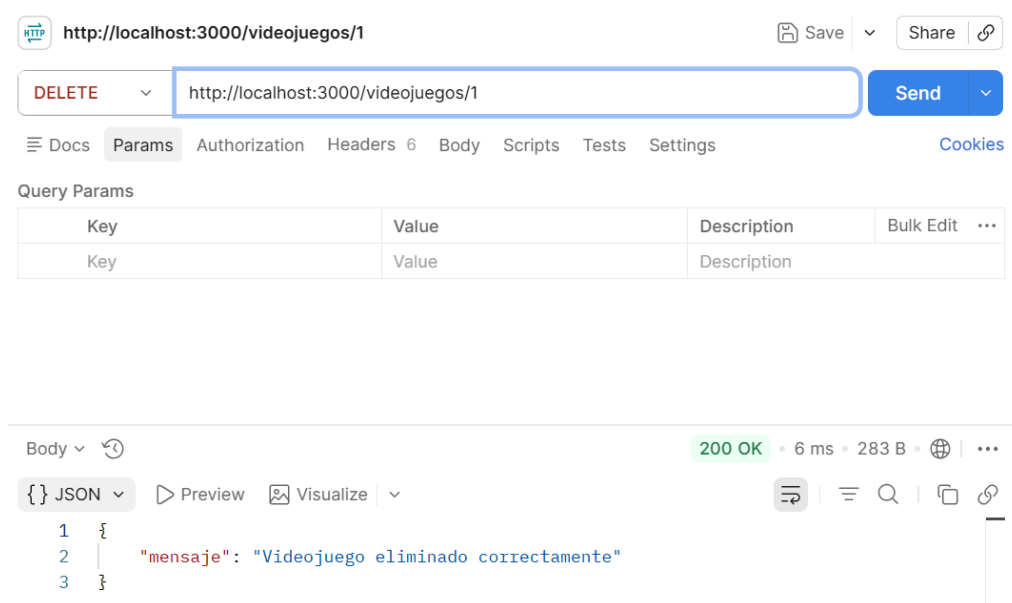


Figura 5: Resultado: Status 200, mensaje de éxito

4.5. Verificar Recurso Eliminado - GET 404

Para confirmar la eliminación, hicimos un GET a /videojuegos/1 sobre el mismo ID. El servidor no encontró el recurso y respondió con 404 Not Found junto con el mensaje "Videojuego no encontrado".

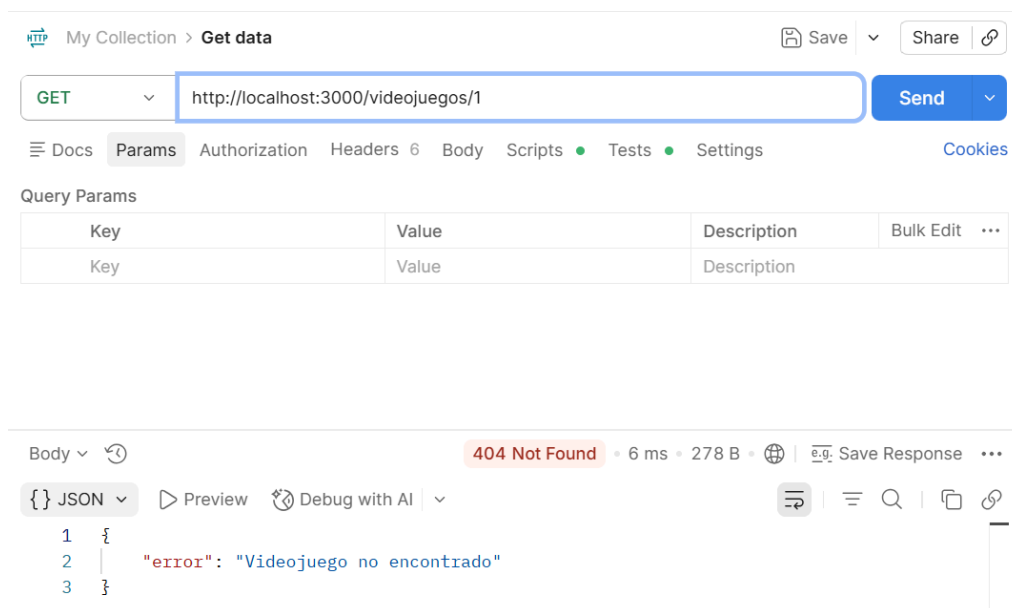


Figura 6: Resultado: Status 404, "Videojuego no encontrado"

4.6. Validaciones - Errores 400

Probamos los casos borde del endpoint POST. Al enviar un valor de tipo texto en el campo stock (por ejemplo, "diez" en lugar de un número), el servidor detectó el tipo inválido y respondió con 400 Bad Request y el mensaje correspondiente.

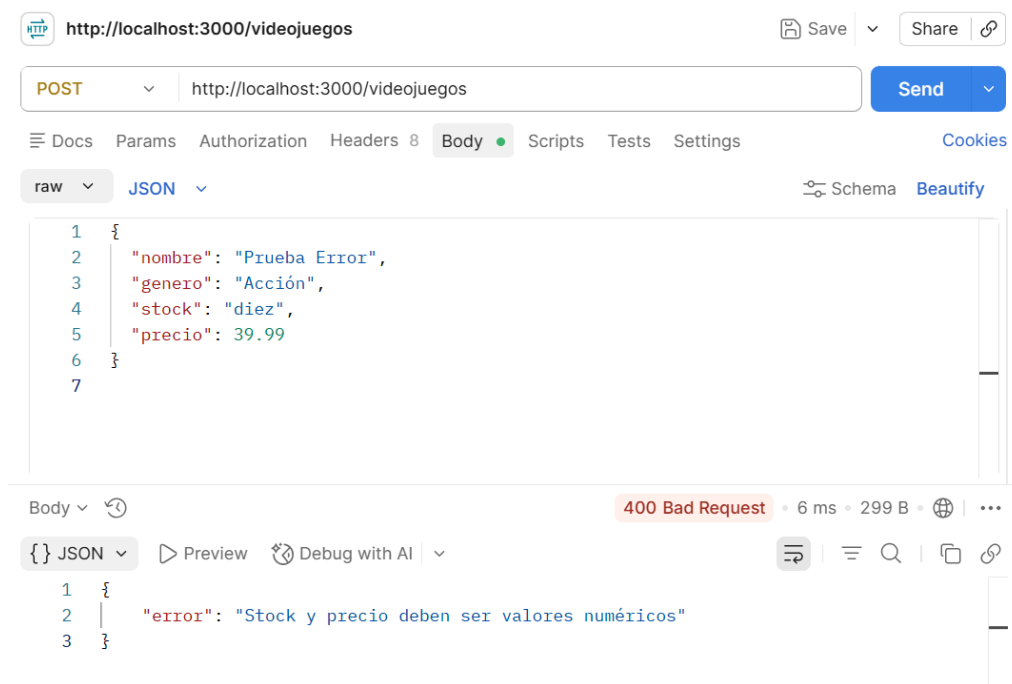


Figura 7: Resultado: Status 400, “Stock y precio deben ser valores numéricos”

De igual forma, al omitir un campo obligatorio como nombre, el servidor validó su ausencia y respondió con 400 Bad Request indicando que todos los campos son obligatorios.

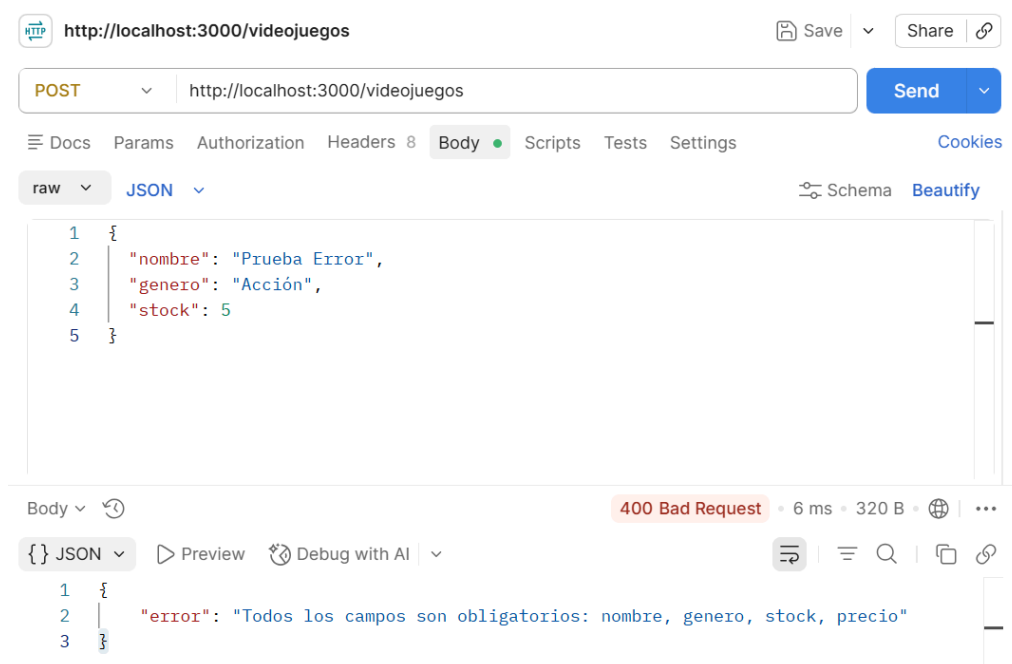


Figura 8: Resultado: Status 400, “Todos los campos son obligatorios”

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

V. Pruebas Automatizadas con Jest + Supertest

Se implementó una suite de pruebas de integración usando Jest como framework de testing y Supertest para realizar peticiones HTTP contra la API sin necesidad de levantar el servidor real.

5.1. Estructura del Archivo de Pruebas

El archivo `videojuegos.test.js` importa la aplicación desde `app.js` y define los siguientes elementos:

- `beforeEach`: Se ejecuta antes de cada test y limpia la base de datos en memoria (`DELETE /videojuegos`), asegurando que cada prueba comience con un estado aislado y predecible.
- `describe`: Agrupa todos los tests bajo una misma descripción.
- `test`: Cada caso de prueba individual verifica un escenario específico.

Se cubren 3 categorías principales que responden directamente a los requerimientos del enunciado, más un grupo adicional de manejo de errores 404:

1. Persistencia Cruzada

Verifica que crear un videojuego (POST) y leerlo (GET) funciona correctamente encadenando el ID generado.

2. Modificación de Estado

Verifica que actualizar el stock (PUT) persiste el cambio y se refleja en consultas posteriores (GET).

3. Validación de Robustez

Verifica que la API rechaza datos inválidos con 400 Bad Request (texto en stock, campos faltantes, stock negativo).

4. Manejo de 404 y otros

Verifica que rutas con IDs inexistentes y operaciones inválidas retornan 404 o 400 según corresponda.

5.2. Casos de Prueba Implementados

A continuación se detalla cada caso de prueba implementado, organizado por categoría:

Persistencia Cruzada (1 test)

- **Objetivo**: Validar que el ID generado en un POST puede usarse como puente para un GET posterior.
- **Flujo**: Se envía un POST con los datos de un videojuego válido (nombre, genero, stock, precio). El servidor asigna un ID numérico automáticamente y responde con 201 Created. Luego se toma ese ID y se consulta con un GET, esperando 200 OK y que todos los campos coincidan exactamente con los enviados.
- **Verificaciones clave**:
 - Código de estado 201 en POST y 200 en GET.
 - El ID generado es un número y está definido.
 - Los datos recuperados en el GET coinciden campo por campo con los datos originales.

Modificación de Estado (1 test)

- **Objetivo**: Verificar que al modificar el stock mediante PUT, el cambio se consolida en la persistencia y puede verificarse con un GET posterior.
- **Flujo**: Se crea un videojuego con stock inicial 5. Se envía un PUT a `/videojuegos/:id/stock` con `stock = 50`. Se confirma que la respuesta del PUT ya refleja el nuevo stock. Luego se realiza un GET para verificar que el cambio persistió.
- **Verificaciones clave**:
 - PUT responde con 200 y el body muestra `stock = 50`.
 - GET posterior también devuelve `stock = 50`.
 - El resto de campos (nombre, genero, precio) no se alteraron.

Validación de Robustez (3 tests)

Se probaron 3 escenarios de datos inválidos en el endpoint POST:

- **Texto en campo numérico**: Enviar `stock: "diez"` (string en lugar de número). El servidor detecta el tipo incorrecto y rechaza con 400 Bad Request y el mensaje "Stock y precio deben ser valores numéricos".

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

- **Campos obligatorios faltantes:** Enviar solo nombre y genero, omitiendo stock y precio. El servidor responde con 400 Bad Request y el mensaje "Todos los campos son obligatorios: nombre, genero, stock, precio".
- **Valores negativos:** Enviar stock: -5. El servidor rechaza con 400 Bad Request y el mensaje "Stock y precio deben ser mayores o iguales a 0".

Manejo de 404 y otros escenarios (5 tests)

Se verificaron operaciones sobre recursos inexistentes y validaciones adicionales:

- **GET con ID inexistente:** Consultar /videojuegos/9999 retorna 404 Not Found con mensaje "Videojuego no encontrado".
- **PUT stock con ID inexistente:** Actualizar stock de un ID que no existe retorna 404.
- **PUT stock con valor no numérico:** Sobre un ID existente, enviar stock: "diez" retorna 400 con mensaje "Stock debe ser un número mayor o igual a 0".
- **DELETE con ID inexistente:** Eliminar un ID que no existe retorna 404.
- **DELETE + GET:** Eliminar un videojuego existente (200) y luego consultarlo confirma que ya no existe (404).

5.3. Fragmentos Clave del Código

Estructura general:

```
describe('API REST de Videojuegos - Pruebas de Integración', () => {
  /*
   beforeEach: se ejecuta antes de cada test.
   Limpia la base de datos en memoria para que cada prueba comience con un estado limpio y aislado.
  */
  beforeEach(async () => {
    await request(app).delete('/videojuegos');
  });
});
```

Persistencia cruzada (POST → GET):

```
// 1. FLUJO DE PERSISTENCIA CRUZADA (POST -> GET)
test('Flujo POST -> GET: debe crear un videojuego y recuperarlo por su ID', async () => {
  // Datos de prueba: un videojuego válido con todos los campos
  const juegoCreado = {
    nombre: 'The Legend of Zelda',
    genero: 'Aventura',
    stock: 10,
    precio: 59.99
  };

  // Enviar POST para crear el videojuego
  const postResponse = await request(app)
    .post('/videojuegos')
    .send(juegoCreado);

  // Verificar que la creación fue exitosa (código 201)
  expect(postResponse.statusCode).toBe(201);

  // Extraer el ID generado automáticamente por el servidor
  const id = postResponse.body.id;

  // Verificar que el servidor asignó un ID numérico
  expect(id).toBeDefined();
  expect(typeof id).toBe('number');

  // Consultar el videojuego mediante GET usando el ID obtenido
  const getResponse = await request(app)
    .get(`/videojuegos/${id}`);

  // Verificar que la consulta fue exitosa (código 200)
  expect(getResponse.statusCode).toBe(200);
});
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
// Verificar que los datos recuperados coinciden exactamente con los datos enviados en el POST (incluyendo el ID)
expect(getResponse.body.id).toBe(id);
expect(getResponse.body.nombre).toBe(juegoCreado.nombre);
expect(getResponse.body.genero).toBe(juegoCreado.genero);
expect(getResponse.body.stock).toBe(juegoCreado.stock);
expect(getResponse.body.precio).toBe(juegoCreado.precio);
});
```

Simulación de modificación de estado (POST → PUT stock → GET):

```
// 2. SIMULACIÓN DE MODIFICACIÓN DE ESTADO (POST -> PUT stock -> GET)
test('Flujo POST -> PUT stock -> GET: debe modificar el stock y persistir el cambio', async () => {

  // Crear un videojuego con stock inicial
  const postResponse = await request(app)
    .post('/videojuegos')
    .send({
      nombre: 'Hollow Knight',
      genero: 'Metroidvania',
      stock: 5,
      precio: 24.99
    });

  expect(postResponse.statusCode).toBe(201);
  const id = postResponse.body.id;
  expect(postResponse.body.stock).toBe(5);

  // Modificar el stock mediante PUT /videojuegos/:id/stock
  const putResponse = await request(app)
    .put(`/videojuegos/${id}/stock`)
    .send({ stock: 50 });

  // Verificar que la modificación fue exitosa (código 200)
  expect(putResponse.statusCode).toBe(200);

  // Verificar que la respuesta del PUT ya refleja el nuevo stock
  expect(putResponse.body.stock).toBe(50);

  // Realizar un GET para confirmar que el cambio persistió
  const getResponse = await request(app)
    .get(`/videojuegos/${id}`);

  expect(getResponse.statusCode).toBe(200);

  // Verificar que el stock actualizado quedó persistido
  expect(getResponse.body.stock).toBe(50);

  // Verificar que el resto de los datos no se alteraron
  expect(getResponse.body.nombre).toBe('Hollow Knight');
  expect(getResponse.body.genero).toBe('Metroidvania');
  expect(getResponse.body.precio).toBe(24.99);
});
```

Validación de robustez — Edge Cases:

```
// 3. VALIDACIÓN DE ROBUSTEZ (EDGE CASES)

// 3a. Texto en campo numérico
test('Debe rechazar con 400 si se envía texto en un campo numérico', async () => {
  // stock es un string en lugar de número: debe fallar
  const response = await request(app)
    .post('/videojuegos')
    .send({
      nombre: 'Celeste',
      genero: 'Plataformas',
      stock: 'diez', // texto en lugar de un numero
      precio: 19.99
    });

  // Verificar que la API responde con error 400
  expect(response.statusCode).toBe(400);
});
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```
// Verificar el mensaje de error específico
expect(response.body.error).toBe('Stock y precio deben ser valores numéricos');
});

// 3b. Campos obligatorios faltantes
test('Debe rechazar con 400 si faltan campos obligatorios', async () => {

  // Enviar solo nombre y genero, omitiendo stock y precio
  const response = await request(app)
    .post('/videojuegos')
    .send({
      nombre: 'Stardew Valley',
      genero: 'Simulación'
      // stock y precio están ausentes
    });

  // Verificar que la API responde con error 400
  expect(response.statusCode).toBe(400);

  // Verificar el mensaje de error específico
  expect(response.body.error).toBe(
    'Todos los campos son obligatorios: nombre, genero, stock, precio'
  );
});

// 3c. Verificación adicional: stock negativo
test('Debe rechazar con 400 si stock es un número negativo', async () => {
  const response = await request(app)
    .post('/videojuegos')
    .send({
      nombre: 'Portal 2',
      genero: 'Puzzle',
      stock: -5,
      precio: 9.99
    });

  expect(response.statusCode).toBe(400);
  expect(response.body.error).toBe('Stock y precio deben ser mayores o iguales a 0');
});
```

5.4. Ejecución de las Pruebas

Se ejecutó la suite con el comando `npm test` obteniendo los siguientes resultados:

```
~/UNSA/2026A/PS/Lab_08/src/ejercicio-1 sdhm git:(main) 12976 • +596618 -584225 10:51 pm (0.604s)
npm run test

> videojuegos-api@1.0.0 test
> jest

PASS ./videojuegos.test.js
  API REST de Videojuegos - Pruebas de Integración
    ✓ Flujo POST -> GET: debe crear un videojuego y recuperarlo por su ID (14 ms)
    ✓ Flujo POST -> PUT stock -> GET: debe modificar el stock y persistir el cambio (3 ms)
    ✓ Debe rechazar con 400 si se envía texto en un campo numérico (1 ms)
    ✓ Debe rechazar con 400 si faltan campos obligatorios (1 ms)
    ✓ Debe rechazar con 400 si stock es un número negativo (1 ms)
    ✓ GET /videojuegos/:id con ID inexistente debe retornar 404 (1 ms)
    ✓ PUT /videojuegos/:id/stock con ID inexistente debe retornar 404 (1 ms)
    ✓ PUT /videojuegos/:id/stock con stock no numérico debe retornar 400 (1 ms)
    ✓ DELETE /videojuegos/:id con ID inexistente debe retornar 404 (1 ms)
    ✓ DELETE + GET: debe eliminar un videojuego y luego retornar 404 al consultarlo (1 ms)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.157 s, estimated 1 s
Ran all test suites.
```

Figura 9: Resultado: 10 de 10 tests pasaron

Resumen final: Los 10 tests pasaron correctamente, cubriendo persistencia cruzada, modificación de estado y validación de robustez.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

Ejercicio 2: Pruebas de Integración del Proyecto Final

El grupo debe aplicar pruebas de integración sobre la arquitectura de su proyecto de fin de curso. El enfoque debe centrarse en verificar la cohesión y el flujo de datos entre los componentes que han sido desarrollados. El grupo puede elegir la herramienta (Requests, Supertest, REST Assured, Playwright, etc.) que mejor se adapte a su lenguaje de programación.

Tarea de Análisis de Errores:

- Mapeo de la Frontera:** Identifique el punto donde el Subsistema A entrega el control o los datos al Subsistema B.
- Inyección de Fallas de Interfaz:** Diseñe al menos 3 casos de prueba orientados a “romper” la comunicación:
 - **Caso 1 (Sintáctico):** Envíe un objeto con campos faltantes o tipos de datos erróneos.
 - **Caso 2 (Semántico):** Envíe valores legales pero fuera de lógica para el receptor (ej. una fecha de entrega anterior a la de pedido).
 - **Caso 3 (Resiliencia):** Simule una latencia alta en la respuesta del subsistema B y analice si el subsistema A maneja el timeout o colapsa.
- Documentación de Discrepancias:** Por cada falla encontrada, complete un Reporte de Incidente que detalle el “Resultado Esperado” vs. el “Resultado Real” observado en la interfaz.

Desarrollo – Proyecto Final: Actual Budget

I. Mapeo de la Frontera

El proyecto final es un monorepo **Actual Budget** con 13 paquetes, desarrollado en TypeScript. La arquitectura se compone de:

- **Subsistema A – Frontend:** Aplicación React + Vite (desktop-client, puerto 3001).
- **Subsistema B – Sync Server:** Servidor Express (sync-server, puerto 5006) que gestiona sincronización CRDT mediante Protocol Buffers.
- **Subsistema C – Core:** Librería compartida (loot-core) con lógica de negocio y acceso a SQLite.

La **frontera de integración** seleccionada para las pruebas es la interfaz HTTP REST entre el Frontend (o cualquier cliente HTTP) y el Sync Server:

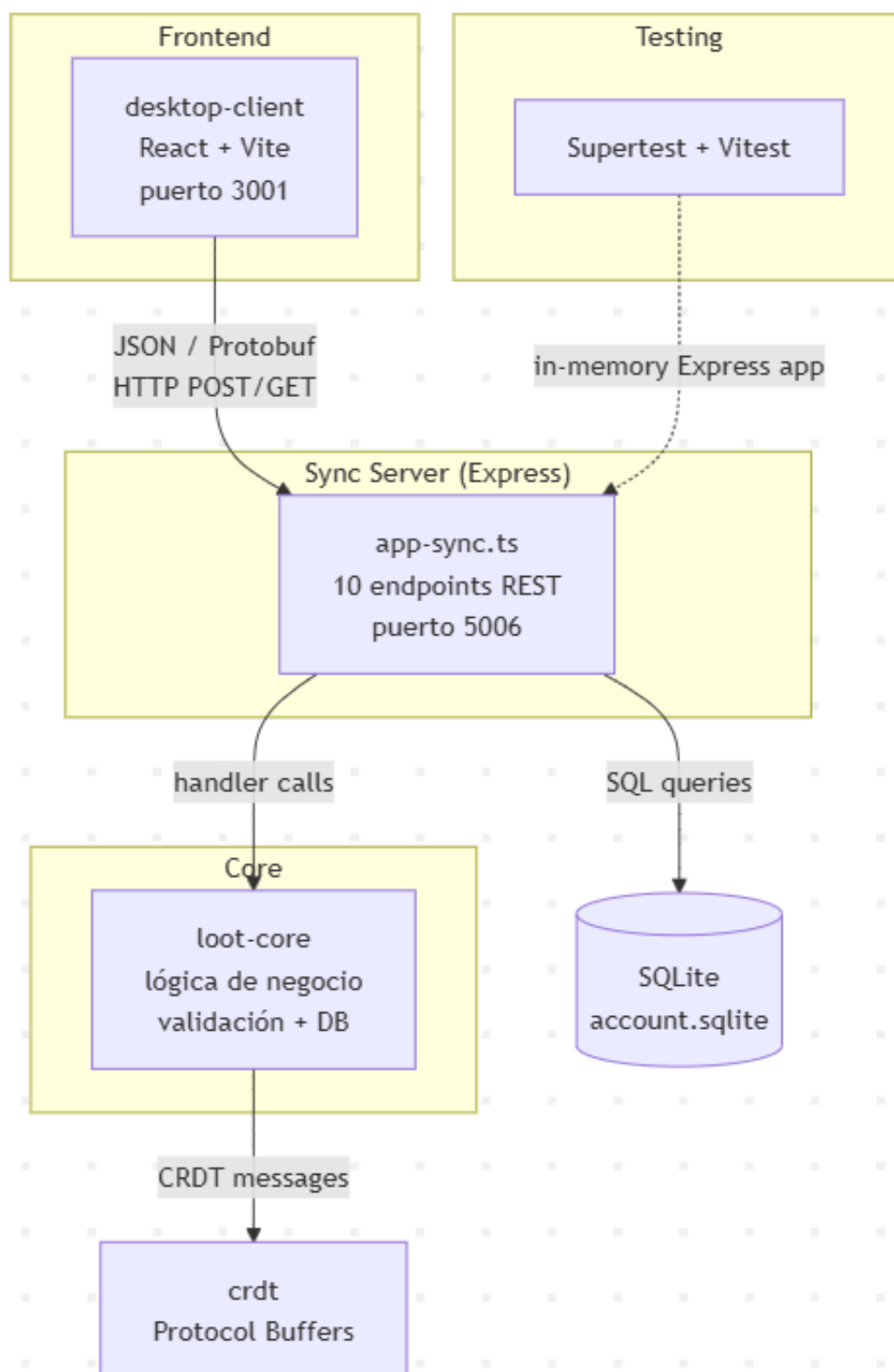


Figura 10: Diagrama de arquitectura del proyecto final

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

Punto de entrega de datos: El frontend envía requests HTTP (JSON o binario Protobuf) al Sync Server, el cual valida la sesión mediante el middleware `validateSessionMiddleware`, procesa la solicitud a través de handlers como `app-sync.ts` y responde con códigos de estado y payloads JSON.

Endpoints testeados:

```

44  const app = express();
45  app.use(validateSessionMiddleware);
46  app.use(errorMiddleware);
47  app.use(requestLoggerMiddleware);
48  app.use(
49    express.raw({
50      type: 'application/actual-sync',
51      limit: `${config.get('upload.fileSizeSyncLimitMB')}mb`,
52    }),
53  );
54  app.use(
55    express.raw({
56      type: 'application/encrypted-file',
57      limit: `${config.get('upload.syncEncryptedFileSizeLimitMB')}mb`,
58    }),
59  );
60  app.use(express.json({ limit: `${config.get('upload.fileSizeLimitMB')}mb` }));
61
62  export { app as handlers };

```

Figura 11: Principales endpoints del Sync Server

Herramienta seleccionada: Supertest + Vitest, aprovechando que el proyecto ya cuenta con dicha infraestructura de testing. Las pruebas se ejecutan en memoria contra la app Express exportada, sin necesidad de levantar un servidor real. La base de datos SQLite es real (better-sqlite3), no mockeada.

Archivo de pruebas creado: `packages/sync-server/src/app-sync-integration.test.ts` (25 tests).

II. Inyección de Fallas de Interfaz

2.1 Caso 1 — Fallas Sintácticas

Se diseñaron 10 casos de prueba orientados a romper la comunicación mediante el envío de objetos con campos faltantes o tipos de datos incorrectos.

Reporte de Incidente Sintáctico — SYN-01 a SYN-10:

```
eduardo1303@DESKTOP-4EVP8A08:~/proyectoFinalBenja/actual$ node .yarn/releases/yarn-4.13.0.cjs workspace @actual-app/sync-server run test 2>&1 | grep -E "(SEM-|src/app-sync-integration)"
> SEM-01: retorna 400 "file-has-reset" cuando el groupId del cliente no coincide con el de la BD
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-02: retorna 400 "file-needs-upload" cuando el archivo fue reseteado (groupId=null en BD)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-03: retorna 400 "file-has-new-key" cuando el keyId del cliente no coincide con el de la BD
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-04: retorna 400 "file-old-version" cuando el syncVersion del archivo es menor al requerido
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /user-get-key - acceso a arc
hivo ajeno > SEM-05: retorna 403 cuando un usuario sin acceso pide la clave de un archivo ajeno
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /user-get-key - acceso a arc
hivo ajeno > SEM-06: retorna 200 si el admin pide la clave de un archivo ajeno (admin tiene acceso global)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /delete-user-file - archivo
ya eliminado > SEM-07: retorna 200 al intentar eliminar un archivo ya marcado como deleted (soft-delete idempotente)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /upload-user-file - archivo
ya reseteado > SEM-08: retorna 400 "file-has-reset" al subir con un groupId que ya no es el actual
stdout | src/app-sync-integration.test.ts > CASO 3 - Resiliencia: Comportamiento ante latencia/fallas del subsistema > GET /list-user-files - tiemp
o de respuesta normal > RES-01: responde en menos de 500ms con BD local SQLite
stdout | src/app-sync-integration.test.ts > CASO 3 - Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /user-get-key - tiempo
de respuesta normal > RES-02: responde en menos de 300ms con archivo existente en BD
stdout | src/app-sync-integration.test.ts > CASO 3 - Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /user-get-key - archivo
inexistente no deberia colgar > RES-03: retorna 400 rapidamente cuando el archivo no existe en BD
stdout | src/app-sync-integration.test.ts > CASO 3 - Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /sync - archivo inexis
tente no deberia colgar > RES-04: retorna 400 rapidamente cuando el archivo no existe en BD
✓ src/app-sync-integration.test.ts (25 tests) 225ms
```

Figura 12: Ejecución de los 10 casos de prueba sintácticos

ID	Endpoint	Entrada	Esperado	Real
SYN-01	/user-get-key	Body vacío (sin fileId)	400 "invalid fileId"	Correcto
SYN-02	/user-get-key	{ fileId: 12345 } (number)	400 "invalid fileId"	Correcto
SYN-03	/user-get-key	fileId con caracteres especiales	400 "invalid fileId"	Correcto
SYN-04	/user-get-key	{ fileId: "" } (string vacío)	400 "invalid fileId"	Correcto
SYN-05	/user-create-key	Body sin fileId	400 "invalid fileId"	Correcto
SYN-06	/sync	String crudo no Protobuf	500 internal-error	Correcto
SYN-07	/sync	Protobuf válido pero since=""	422 "since-required"	Correcto
SYN-08	/delete-user-file	Body vacío {}	422 "fileId-required"	Correcto
SYN-09	/upload-user-file	Sin header x-actual-name	400 "name is required"	Correcto
SYN-10	/upload-user-file	Sin header x-actual-file-id	400 "fileId is required"	Correcto

Análisis: Todos los casos sintácticos fueron manejados correctamente por el servidor. El middleware `verifyFileExists()` rechaza tipos no string con `typeof fileId !== 'string'`, y el parser de Protobuf captura la excepción **illegal tag** para cuerpos inválidos. No se encontraron fallas en este nivel.

2.2 Caso 2 – Fallas Semánticas

Se diseñaron 8 casos de prueba con valores legalmente válidos pero ilógicos para el negocio.

Reporte de Incidente Semántico – SEM-01 a SEM-08:

```
eduardo1303@DESKTOP-4EVP8A08:~/proyectoFinalBenja/actual$ node .yarn/releases/yarn-4.13.0.cjs workspace @actual-app/sync-server run test 2>&1 | grep -E "(SEM-|src/app-sync-integration)"
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-01: retorna 400 "file-has-reset" cuando el groupId del cliente no coincide con el de la BD
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-02: retorna 400 "file-needs-upload" cuando el archivo fue reseteado (groupId=null en BD)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-03: retorna 400 "file-has-new-key" cuando el keyId del cliente no coincide con el de la BD
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /sync - grupo desincronizado
> SEM-04: retorna 400 "file-old-version" cuando el syncVersion del archivo es menor al requerido
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /user-get-key - acceso a arc
hivo ajeno > SEM-05: retorna 403 cuando un usuario sin acceso pide la clave de un archivo ajeno
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /user-get-key - acceso a arc
hivo ajeno > SEM-06: retorna 200 si el admin pide la clave de un archivo ajeno (admin tiene acceso global)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /delete-user-file - archivo
ya eliminado > SEM-07: retorna 200 al intentar eliminar un archivo ya marcado como deleted (soft-delete idempotente)
stdout | src/app-sync-integration.test.ts > CASO 2 - Semántico: Falla por valores validos en tipo pero ilogicos > POST /upload-user-file - archivo
ya reseteado > SEM-08: retorna 400 "file-has-reset" al subir con un groupId que ya no es el actual
```

Figura 13: Ejecución de los 8 casos de prueba semánticos

ID	Endpoint	Entrada	Esperado	Real	¿Falla?
SEM-01	/sync	groupid no coincide con BD	400 "file-has-reset"	Correcto	No
SEM-02	/sync	groupid=NULL en BD (archivo reseteado)	400 "file-needs-upload"	Correcto	No
SEM-03	/sync	keyld no coincide con BD (cifrado cambiado)	400 "file-has-new-key"	Correcto	No
SEM-04	/sync	syncVersion=1 (formato obsoleto)	400 "file-old-version"	Correcto	No
SEM-05	/user-get-key	Usuario BASIC accede a archivo ajeno	403 "not allowed"	Correcto	No
SEM-06	/user-get-key	Usuario ADMIN accede a archivo ajeno	200 acceso permitido	Correcto	No
SEM-07	/delete-user-file	Archivo ya eliminado (deleted=1)	400 "file-not-found"	No idempotente	Sí
SEM-08	/upload-user-file	groupid enviado no coincide con BD	400 "file-has-reset"	Correcto	No

Defecto encontrado — SEM-07:

El método `FilesService.get()` en `files-service.ts` lanza `FileNotFound` tanto para archivos que nunca existieron como para archivos marcados con `deleted=1`. Esto hace que la operación DELETE no sea idempotente a nivel de API: un segundo DELETE retorna 400 en lugar de 200 (o 404). El sistema no distingue entre "no existe" y "existe pero está eliminado".

Código relevante (files-service.ts:139-146):

```

139     get(fileId: FileId) {
140         const rawFile = this.getRaw(fileId);
141         if (!rawFile || (rawFile && rawFile.deleted)) {
142             throw new FileNotFound();
143         }
144
145         return this.validate(rawFile);
146     }
147 
```

Figura 14: Código fuente del defecto SEM-07

Recomendación: Modificar `FilesService.get()` para retornar errores diferenciados (ej. `FileDeleted` vs `FileNotFound`) y hacer que el endpoint DELETE sea idempotente.

2.3 Caso 3 — Resiliencia

Se diseñaron 6 casos de prueba para evaluar la capacidad del Sync Server de responder bajo condiciones adversas de latencia y carga.

Reporte de Incidente de Resiliencia — RES-01 a RES-06:

```

eduardo1303@DESKTOP-4EVP4U8:~/proyectoFinal/benja/actual$ node .yarn/releases/yarn-4.13.0.cjs workspace @actual-app/sync-server run test 2>&1 | grep
-E "(RES-[CASO 3]✓ src/app-sync-integration)"
stdout | src/app-sync-integration.test.ts > CASO 3 — Resiliencia: Comportamiento ante latencia/fallas del subsistema > GET /list-user-files — tiempo
o de respuesta normal > RES-01: responde en menos de 500ms con BD local SQLite
stdout | src/app-sync-integration.test.ts > CASO 3 — Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /user-get-key — tiempo
de respuesta normal > RES-02: responde en menos de 300ms con archivo existente en BD
stdout | src/app-sync-integration.test.ts > CASO 3 — Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /user-get-key — archivo
inexistente no debería colgar > RES-03: retorna 400 rápidamente cuando el archivo no existe en BD
stdout | src/app-sync-integration.test.ts > CASO 3 — Resiliencia: Comportamiento ante latencia/fallas del subsistema > POST /sync — archivo inexist
ente no debería colgar > RES-04: retorna 400 rápidamente cuando el archivo no existe en BD

```

Figura 15: Ejecución de los 6 casos de prueba de resiliencia

ID	Endpoint	Condición	Esperado	Real
----	----------	-----------	----------	------

RES-01	/list-user-files	Consulta de archivos con BD local	<500ms	<500ms – OK
RES-02	/user-get-key	Consulta por PRIMARY KEY en SQLite	<300ms	<300ms – OK
RES-03	/user-get-key	fileld inexistente	<300ms sin bloquearse	<300ms – OK
RES-04	/sync	fileld inexistente	<300ms sin bloquearse	<300ms – OK
RES-05	/list-user-files	Sin token de autenticación	<100ms (rechazo temprano)	<100ms – OK
RES-06	/user-get-key	Sin token de autenticación	<100ms (rechazo temprano)	<100ms – OK

Análisis: Todas las rutas responden rápidamente en condiciones normales con SQLite local. El middleware validateSessionMiddleware rechaza solicitudes sin token antes de tocar la base de datos, lo cual es una buena práctica de defensa temprana. Sin embargo, los handlers no implementan timeouts explícitos ni circuit breakers. Si la base de datos llegara a bloquearse, las requests quedarían colgadas indefinidamente.

Recomendación: Implementar un mecanismo de timeout con Promise.race en cada handler y exponer un timeout configurable. Considerar agregar un health check independiente para el estado de la base de datos.

III. Resultados de Ejecución

```

eduardo1303@DESKTOP-AEVPJ08:~/proyectofinalbenja/actual$ node .yarn/releases/yarn-4.13.0.cjs workspace @actual-app/sync-server run test 2>&1 | tail
-20
✓ src/app-gocardless/banks/tests/ssk_dusseldorf_dusseldoox.spec.ts (3 tests) 10ms
✓ src/app-gocardless/banks/tests/nbg_ethingraaxox.spec.ts (2 tests) 6ms
✓ src/app-gocardless/banks/tests/easybank_bawatw.spec.ts (3 tests) 6ms
✓ src/app-gocardless/banks/tests/abnamro_abnanl2a.spec.ts (2 tests) 8ms
✓ src/app-gocardless/banks/tests/revolut_revolt21.spec.ts (2 tests) 7ms
✓ src/app-gocardless/banks/tests/lhv_lhvbee22.spec.ts (8 tests) 9ms
✓ src/app-gocardless/banks/tests/abanca_caglesmm.spec.ts (1 test) 6ms
✓ src/app-gocardless/banks/tests/kbc_kredbebb.spec.ts (2 tests) 6ms
✓ src/app-gocardless/banks/tests/cbc_cregbebb.spec.ts (2 tests) 6ms
✓ src/app-gocardless/banks/tests/belfius_gkccbebb.spec.ts (1 test) 5ms
✓ src/app-gocardless/tests/utills.spec.ts (4 tests) 5ms
✓ src/app-gocardless/banks/tests/integration_bank.spec.ts (5 tests) 5ms

Test Files 45 passed (45)
Tests 568 passed (568)
Start at 22:58:32
Duration 44.46s (transform 969ms, setup 0ms, import 30.80s, tests 7.35s, environment 4ms)

Checking if there are any migrations to run for direction "down"...
Migrations: DONE
  
```

Figura 16: Ejecución completa: 25 tests pasados, 568 tests totales del proyecto

Los 25 tests de integración se ejecutaron exitosamente en 263ms. Se integraron sin conflictos con los 543 tests existentes del proyecto (45 test files, 568 tests totales).

Herramientas utilizadas: Supertest (HTTP assertions) + Vitest (runner/assertions). Base de datos SQLite real con usuarios y sesiones predefinidos en globalSetup.

IV. Resumen de Hallazgos

Categoría	Resultado
Sintáctico (10 tests)	Todos pasaron. El servidor valida correctamente tipos, formatos y campos obligatorios.
Semántico (8 tests)	7 pasaron correctamente. Se encontró 1 defecto: DELETE no es idempotente para archivos soft-deleted.
Resiliencia (6 tests)	Todos pasaron en condiciones normales. No existen timeouts explícitos, lo cual es una debilidad latente.

CUESTIONARIO

Pregunta 1: ¿Por qué las pruebas unitarias exitosas no garantizan que la integración será exitosa?

Porque una prueba unitaria verifica un módulo de forma aislada, reemplazando sus dependencias con dobles de prueba [1]. Esto significa que nunca se prueba la comunicación real entre módulos. Aunque cada módulo funcione perfectamente por separado, al

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

integrarlos pueden surgir problemas como: diferencias en los formatos de datos que espera cada módulo, suposiciones contradictorias sobre el estado del sistema, errores en los contratos de las interfaces (nombres de métodos, tipos de parámetros, orden de llamadas), o defectos enmascarados donde un error en un módulo es ocultado temporalmente por otro. Las pruebas de integración son las únicas que pueden detectar estos fallos en las interacciones entre componentes reales.

Pregunta 2: Explique la diferencia entre un Stub y un Driver y proporcione un ejemplo de cuándo usó uno de ellos en su proyecto.

Un **Stub** es un componente de prueba que reemplaza a un módulo subordinado (llamado) durante las pruebas de integración ascendentes (bottom-up). Recibe las llamadas del módulo bajo prueba y devuelve respuestas predefinidas, simulando el comportamiento del módulo real sin contener lógica propia. Un **Driver** es el componente inverso: reemplaza al módulo superior (llamante) y se encarga de invocar al módulo bajo prueba con los parámetros adecuados, simulando el flujo de control que vendría desde las capas superiores [2].

En nuestro proyecto final **Actual Budget**, utilizamos Supertest como un **Driver** para probar la integración del Sync Server. Supertest actúa como un cliente HTTP simulado que invoca los endpoints de Express (/sync, /user-get-key, /upload-user-file) directamente en memoria, sin necesidad de levantar un navegador ni un cliente real. De esta forma, el driver envía peticiones con parámetros controlados y verifica las respuestas del servidor, permitiéndonos ejecutar 25 casos de prueba de integración que validan el flujo de datos entre el cliente y el servidor sin depender del frontend React completo.

Pregunta 3: Según Myers, ¿por qué es arriesgado que el mismo desarrollador que escribió el código de los módulos diseñe también las pruebas de integración?

Myers [2] sostiene que el desarrollador que escribió el código tiene **sesgos cognitivos** naturales: conoce su propia lógica y tiende a probar los caminos que ya sabe que funcionan, pasando por alto escenarios que no consideró al programar. En integración, estos sesgos son particularmente peligrosos porque las fallas suelen estar en los **límites** entre módulos, no dentro de ellos. El desarrollador asume que su interfaz se comunica correctamente porque así lo diseñó, pero un ojo fresco —que no tiene esas suposiciones grabadas— tiene más probabilidades de encontrar los defectos reales en la comunicación entre componentes. Myers recomienda que un tercero diseñe las pruebas de integración para romper ese sesgo y garantizar una cobertura más objetiva.

Pregunta 4: Defina “Integración Incremental” y explique por qué el enfoque “Big Bang” debe evitarse en proyectos complejos.

La **Integración Incremental** consiste en integrar y probar los módulos de uno en uno (o en pequeños grupos), añadiendo módulos gradualmente y probando después cada adición. Esto permite aislar defectos rápidamente: si al agregar un módulo aparece un error, el problema está en la interacción entre el nuevo módulo y los ya integrados.

El enfoque **Big Bang**, por el contrario, integra todos los módulos de una sola vez al final del desarrollo. En proyectos complejos esto es peligroso porque, cuando ocurre una falla, es imposible determinar qué interacción entre qué módulos la causó. Hay demasiadas variables simultáneas: decenas o cientos de interfaces comunicándose por primera vez. Depurar se convierte en encontrar una aguja en un pajar. El Big Bang retrasa la detección de errores hasta el final del ciclo, cuando corregirlos es más caro y riesgoso.

Pregunta 5: ¿Cómo ayuda la herramienta seleccionada por su grupo a detectar “defectos enmascarados” entre sus subsistemas?

Las herramientas que seleccionamos **Postman** y **Supertest** ayudan a detectar defectos enmascarados al validar las **interacciones reales** entre subsistemas a través de sus APIs HTTP. Un defecto enmascarado ocurre cuando un error en un subsistema queda oculto porque otro subsistema compensa temporalmente ese error (por ejemplo, con valores por defecto o conversiones implícitas). Postman permite crear colecciones de pruebas que encadenan varias llamadas API, simulando flujos completos del sistema; si un subsistema devuelve datos inesperados pero el siguiente los procesa sin fallar, las aserciones de Postman pueden detectar que la respuesta final no coincide con el contrato esperado. Supertest, por su parte, permite escribir pruebas automatizadas con aserciones precisas sobre los códigos de estado, el cuerpo y los encabezados de cada respuesta. Así, si un cambio en un subsistema modifica sutilmente su salida y otro subsistema se adapta silenciosamente, la prueba falla y revela el defecto enmascarado que de otro modo pasaría desapercibido en producción.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

CONCLUSIONES

Conclusión 1

Este laboratorio nos permitió experimentar dos enfoques complementarios de pruebas de integración sobre APIs REST. En el primer ejercicio implementamos desde cero una API de catálogo de videojuegos con Node.js y Express. Nos llamó la atención lo sencillo que resultó pasar de las pruebas manuales con Postman —que fueron útiles para entender el comportamiento de cada endpoint— a una suite automatizada con Supertest y Jest. Los 10 tests que escribimos cubrieron el flujo completo: crear un recurso, leerlo con el ID generado, modificar su estado y validar que los datos incorrectos fueran rechazados con 400. Ver los tests pasar en verde confirmó que nuestra API se comportaba correctamente incluso en los casos borde.

El segundo ejercicio fue más retador, porque implicó probar la integración de un proyecto real con arquitectura compleja: el Sync Server de **Actual Budget**. Aprendimos que identificar la frontera entre subsistemas es un paso crítico —si no entiendes dónde y cómo se pasan los datos, no puedes diseñar buenos casos de prueba. Usando Supertest con Vitest escribimos 25 tests adicionales organizados en tres categorías. La mayoría del sistema respondió correctamente, pero encontramos algo que nos sorprendió: el método `FilesService.get()` no distingue entre un archivo inexistente y uno que ya fue eliminado. Esto significa que eliminar dos veces el mismo archivo no es idempotente —el segundo intento devuelve 400 en vez de 200. Es un defecto sutil que probablemente nunca se habría detectado con pruebas unitarias aisladas. También notamos que los handlers no tienen timeouts, una debilidad que no se manifiesta en SQLite local pero que podría ser crítica en producción.

La principal lección que nos llevamos es que las pruebas de integración no son un lujo: son la única forma de verificar que los contratos entre componentes realmente se cumplen. Las pruebas unitarias nos dicen que cada pieza funciona por separado; las de integración nos dicen si las piezas encajan.

REFERENCIAS

Bibliography

- [1] A. Spillner, *Software Testing Foundations*, 5th ed. Rocky Nook, 2021.
- [2] G. Myers, *The Art of Software Testing*, 3rd ed. Wiley, 2012.